

Authors: Essa Janneh & Syed Umair Anwar

Date: February 3, 2026

Institution: University of Florence - Software Architectures and Methodologies (SWAM)

Table of Contents

- 1. Application Context and Problem Statement
 - 1.1 Operative Domain/Context
 - 1.2 Target Users and Key Needs
 - 1.3 Problem Statement
- 2. Requirements Analysis
 - 2.1 Functional Requirements
 - 2.2 Non-Functional Requirements
- 3. Conceptual Domain Model
 - 3.1 UML Class Diagram
 - 3.2 Entity Descriptions
- 4. Use Case Analysis
 - 4.1 Use Case: UC1 - User Registration
 - 4.2 Use Case: UC2 - Register Vehicle & Purchase Parking Pass
 - 4.3 Use Case: UC3 - Record Violation & Generate Report
- 5. Architectural Considerations
 - 5.1 Layered Architecture Overview
 - 5.2 Layer Responsibilities
 - 5.3 RESTful Architecture Principles
 - 5.4 UML Deployment Diagram
- 6. Implementation
 - 6.1 Backend (Java/Jakarta EE) Architecture
 - 6.2 Frontend (Angular) Architecture
- 7. Summary
 - 7.1 Key Architectural Achievements:
 - 7.2 Technical Excellence:

1. Application Context and Problem Statement

1.1 Operative Domain/Context

The **Municipality Motor Vehicle System** operates within the domain of municipal vehicle management and traffic enforcement. This system is designed to modernize the

administrative processes of a municipality responsible for managing vehicle registration, parking permits, and traffic violations across multiple districts and parking zones.

The application domain encompasses:

- **Vehicle Registration & Ownership Management:** Citizens can register and manage their personal vehicles (cars and motorcycles) with various engine types (Electric, Hybrid, Internal Combustion Engine).
- **Parking Permit Administration:** The system issues and manages parking passes for specific zones within the municipality, with time-based validity periods.
- **Traffic Violation Tracking & Reporting:** Law enforcement officers record and track parking violations and vehicle infractions in real-time.
- **District & Zone Organization:** The municipality is organized into multiple districts (D1, D2, D3, D4, D5) and parking zones (Z1, Z2, etc.) for better traffic management and enforcement.

1.2 Target Users and Key Needs

The system serves three distinct user roles, each with specific needs:

1.2.1 Citizens (Residents)

- **Need:** Manage personal vehicle records and purchase parking permits for convenient urban mobility
- **Key Activities:**
 - Register and maintain multiple vehicles with accurate details (license plate, model, color, engine type)
 - Purchase and track active parking passes
 - View personal vehicle and pass information
 - Understand their violation history if applicable

1.2.2 Law Enforcement Officers

- **Need:** Efficiently enforce parking regulations and record violations in real-time
- **Key Activities:**
 - Verify vehicle registration and parking permit validity using license plates
 - Record traffic violations with accurate location (district) and time stamps
 - Access violation statistics and enforcement metrics
 - Support municipality-wide traffic compliance monitoring

1.2.3 System Administrators

- **Need:** Maintain system integrity and manage master data for the entire municipality
- **Key Activities:**
 - Configure and manage districts and parking zones
 - Manage vehicle types and engine classifications
 - Generate comprehensive reports on violations and system usage
 - Monitor system performance and data consistency
 - Ensure role-based access control and data security

1.3 Problem Statement

The Municipality faces the following challenges with its current vehicle management infrastructure:

1. **Inefficient Manual Processes:** Vehicle registration and permit issuance are handled through manual paper-based systems, leading to delays and errors.
2. **Lack of Real-time Visibility:** Law enforcement officers have no immediate access to vehicle registration status or valid parking permits, making violation verification time-consuming and error-prone.
3. **Fragmented Data Management:** Violation records, vehicle registrations, and parking permits are maintained in separate systems with no integration, preventing comprehensive reporting and analysis.
4. **Scalability Issues:** The current system cannot efficiently handle the growing number of vehicles and violations as the city expands.
5. **Reporting Limitations:** Administrators cannot quickly generate reports on violation patterns, vehicle distribution by type/engine, or enforcement metrics needed for policy decisions.

Solution Proposed: The Municipality Motor Vehicle System is a comprehensive, cloud-ready web application that:

- Centralizes all vehicle and permit data in a unified, secure database
- Provides real-time access to critical information for enforcement and administration
- Enables instant violation recording and reporting
- Supports data-driven decision-making through comprehensive analytics and PDF reports
- Scales efficiently to handle growing municipal needs

2. Requirements Analysis

2.1 Functional Requirements

The system implements the following core functional requirements:

2.1.1 Authentication & Authorization

- **FR1:** The system SHALL allow citizens, law enforcement officers, and administrators to register new user accounts with unique usernames and email addresses.
- **FR2:** The system SHALL provide secure login functionality that authenticates users based on username and password credentials.
- **FR3:** The system SHALL enforce role-based access control (RBAC) with three distinct roles: CITIZEN, LAW_ENFORCEMENT, and ADMIN, restricting access to role-appropriate features.

2.1.2 *Vehicle Management*

- **FR4:** The system SHALL allow citizens to register new vehicles with the following attributes: license plate (unique), model, color, vehicle type (CAR/MOTORBIKE), and engine type (ELECTRIC/HYBRID/ICE).
- **FR5:** The system SHALL enable citizens to view all their registered vehicles in a dashboard interface.
- **FR6:** The system SHALL allow citizens to update vehicle information (model, color) while maintaining license plate immutability.
- **FR7:** The system SHALL allow citizens to delete vehicles from their personal inventory.
- **FR8:** The system SHALL provide vehicle search and filtering capabilities by userId, license plate, and vehicle type.

2.1.3 *Parking Permit Management*

- **FR9:** The system SHALL allow citizens to purchase parking passes for specific zones with a validity period (default: 1 month from purchase).
- **FR10:** The system SHALL associate up to 4 license plates per parking pass, allowing a single pass to cover multiple vehicles.
- **FR11:** The system SHALL prevent duplicate active passes for the same zone and user (a user cannot have two active passes for the same zone simultaneously).
- **FR12:** The system SHALL provide pass verification by license plate, returning validity status and expiry date.
- **FR13:** The system SHALL enable citizens to view all their active and expired parking passes with associated license plates.
- **FR14:** The system SHALL automatically calculate and display pass expiration dates.

2.1.4 *Violation Management & Recording*

- **FR15:** The system SHALL allow law enforcement officers to record traffic violations with the following information: timestamp, district, vehicle type, engine type, and license plate.
- **FR16:** The system SHALL provide a simulation feature for testing violation recording and enforcement logic.
- **FR17:** The system SHALL store violation records with accurate date-time stamps for audit and reporting purposes.
- **FR18:** The system SHALL enable querying violations by license plate to support enforcement operations.

2.1.5 *Reporting & Analytics*

- **FR19:** The system SHALL generate violation reports grouped by district, vehicle type, or engine type.
- **FR20:** The system SHALL provide time-period-based filtering for reports (all time, last day, last week, last month).
- **FR21:** The system SHALL generate professional PDF reports containing violation statistics, municipality information, and generated timestamps.
- **FR22:** The system SHALL provide real-time counters for active parking passes and total violations.

2.1.6 *Data Management*

- **FR23:** The system SHALL automatically seed the database with predefined districts (D1, D2, D3, D4, D5) on first application startup.
- **FR24:** The system SHALL automatically seed the database with predefined parking zones (Z1, Z2, Z3, Z4) associated with appropriate districts.
- **FR25:** The system SHALL provide administrative endpoints to retrieve all available districts, zones, vehicle types, and engine types.
- **FR26:** The system SHALL ensure master data consistency across all user registrations and vehicle records.

2.2 Non-Functional Requirements

2.2.1 *Architectural Style & Design Patterns*

- **NFR1:** The system SHALL be designed using a RESTful architecture adhering to REST principles: statelessness, resource-oriented design, standard HTTP methods (GET, POST, PUT, DELETE), and JSON-based resource representation.
- **NFR2:** The system SHALL implement a layered architecture with clear separation of concerns: Presentation Layer (Angular Frontend), Business Logic Layer (JAX-RS Resources), Data Access Layer (JPA/Hibernate), and Data Storage Layer (PostgreSQL).
- **NFR3:** The system SHALL employ the Data Transfer Object (DTO) pattern to decouple internal entity models from external API contracts.
- **NFR4:** The system SHALL implement the Mapper pattern for clean, maintainable entity-to-DTO conversions and vice versa.
- **NFR5:** The system SHALL implement the Repository pattern implicitly through EntityManager for database operations.

2.2.3 *Security Requirements*

- **NFR6:** The system SHALL implement Cross-Origin Resource Sharing (CORS) to allow secure communication between the Angular frontend (port 4200) and WildFly backend (port 8080).
- **NFR7:** The system SHALL store passwords in plaintext for development purposes (note: production deployment requires password hashing with bcrypt or similar algorithms).
- **NFR8:** The system SHALL implement role-based access control (RBAC) to restrict sensitive operations (like back-office admin features) to authorized users only.
- **NFR9:** The system SHALL validate all user input to prevent SQL injection and cross-site scripting (XSS) attacks.
- **NFR10:** The system SHALL use transactional boundaries (Jakarta Transactions) to ensure data consistency and atomicity of multi-step operations.

2.2.4 *Performance & Scalability*

- **NFR11:** The system SHALL implement eager loading with “Join Fetch” queries to prevent N+1 query problems and lazy-loading issues.
- **NFR12:** The system SHALL support concurrent user sessions without performance degradation.
- **NFR13:** The system SHALL scale horizontally by allowing deployment across multiple WildFly instances behind a load balancer.
- **NFR14:** The system SHALL support database connection pooling for efficient resource utilization.
- **NFR15:** The system SHALL handle pagination for large result sets (e.g., extensive violation records) to minimize memory consumption.

2.2.5 *Usability & User Experience*

- **NFR16:** The Frontend SHALL provide a modern, responsive user interface with a dark-themed sidebar and intuitive navigation.
- **NFR17:** The Frontend SHALL display real-time dashboard data with automatic refresh intervals (e.g., every 5 seconds) for live violation updates.
- **NFR18:** The Frontend SHALL provide clear error messages and validation feedback for user actions.
- **NFR19:** The Frontend SHALL support form-based data entry with client-side validation before server submission.

2.2.6 *Reliability & Maintainability*

- **NFR20:** The system SHALL implement proper error handling and logging throughout the backend to facilitate debugging and monitoring.
- **NFR21:** The system SHALL use standardized HTTP status codes (200, 201, 400, 401, 404, 500) in all API responses.
- **NFR22:** The system SHALL provide graceful error responses with meaningful error messages in JSON format.
- **NFR23:** The system SHALL implement transaction rollback mechanisms to prevent partial data updates during error conditions.
- **NFR24:** The system SHALL be designed to support easy deployment to new workstations using relative path configuration in deployment scripts.

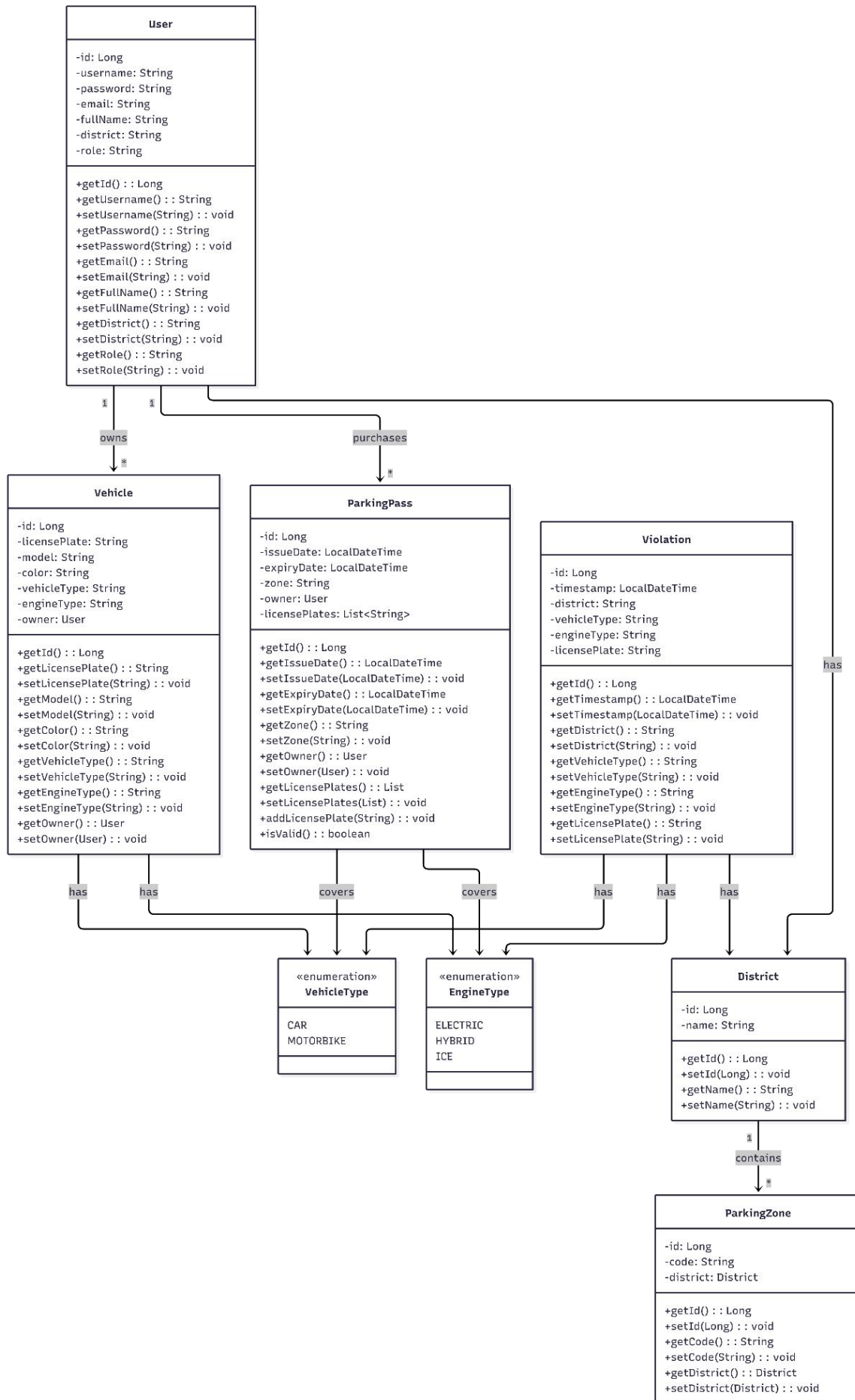
2.2.7 *Data Integrity & Consistency*

- **NFR25:** The system SHALL maintain referential integrity through foreign key constraints in the database.
 - **NFR26:** The system SHALL use unique constraints on critical fields (username, email, license plate) to prevent data duplication.
 - **NFR27:** The system SHALL implement timestamp tracking for audit purposes on violation records.
 - **NFR28:** The system SHALL ensure atomic operations for complex workflows (e.g., vehicle registration + pass purchase) using database transactions.
-

3. Conceptual Domain Model

3.1 UML Class Diagram

The system's conceptual domain model consists of the following core entities and their relationships:



3.2 Entity Descriptions

3.2.1 User

- **Purpose:** Represents system users with distinct roles and permissions
- **Key Attributes:**
 - username: Unique identifier for user login (must be unique)
 - password: Authentication credential (plaintext in development)
 - email: Contact information and unique identifier
 - fullName: User's legal name
 - district: Municipality district where the user resides (D1-D5)(Foreign key reference to District Table)
 - role: CITIZEN, LAW_ENFORCEMENT, or ADMIN - determines system access level
- **Relationships:**
 - One-to-Many with Vehicle (a user can own multiple vehicles)
 - One-to-Many with ParkingPass (a user can purchase multiple passes)
 - Implicitly referenced in Violation records for enforcement tracking

3.2.2 Vehicle

- **Purpose:** Represents motor vehicles registered by citizens within the municipality
- **Key Attributes:**
 - licensePlate: Unique vehicle identifier in the real world (must be unique in system)
 - model: Vehicle model/brand description
 - color: Vehicle color for identification
 - vehicleType: CAR or MOTORBIKE classification for regulation differentiation
 - engineType: ELECTRIC, HYBRID, or ICE (Internal Combustion Engine) for environmental compliance tracking
 - owner: Foreign key reference to owning User
- **Relationships:**
 - Many-to-One with User (many vehicles can be owned by one user)
 - Referenced by ParkingPass (via licensePlates collection)
 - Implicitly referenced in Violation records

3.2.3 ParkingPass

- **Purpose:** Represents time-limited parking permits for specific zones
- **Key Attributes:**
 - issueDate: Timestamp when pass was purchased
 - expiryDate: Timestamp when pass validity ends (typically 1 month from issue)
 - zone: Specific parking zone code (Z1, Z2, Z3, Z4) where pass is valid
 - licensePlates: Collection of up to 4 license plates covered by this single pass

- owner: Foreign key reference to User who purchased the pass
- **Relationships:**
 - Many-to-One with User (a user can have multiple passes for different zones)
 - Implicitly referenced in pass validation logic
- **Business Logic:** Users cannot have two active passes for the same zone simultaneously

3.2.4 Violation

- **Purpose:** Records traffic infractions and parking violations for enforcement and reporting
- **Key Attributes:**
 - timestamp: Precise date-time when violation was recorded
 - district: Municipality district where violation occurred (D1-D5)
 - vehicleType: Type of vehicle involved (CAR/MOTORBIKE)
 - engineType: Engine type of vehicle involved (ELECTRIC/HYBRID/ICE)
 - licensePlate: Vehicle license plate involved in violation
- **Relationships:**
 - No direct foreign key to User (violations are recorded independently by law enforcement)
 - Referenced by reporting queries for analytics and compliance metrics
- **Audit Trail:** Maintains permanent record of all enforcement activities

3.2.5 District

- **Purpose:** Master data representing municipality administrative divisions
- **Key Attributes:**
 - id: Unique identifier
 - name: District code (D1, D2, D3, D4, D5)
- **Relationships:**
 - One-to-Many with ParkingZone (each district contains multiple parking zones)
- **Immutability:** Districts are seeded on startup and typically not modified

3.2.6 ParkingZone

- **Purpose:** Master data representing specific parking areas within districts
- **Key Attributes:**
 - id: Unique identifier
 - code: Zone code (Z1, Z2, Z3, Z4)
 - district: Reference to parent District
- **Relationships:**
 - Many-to-One with District (multiple zones per district)
- **Usage:** Referenced by ParkingPass for zone-specific permit validation

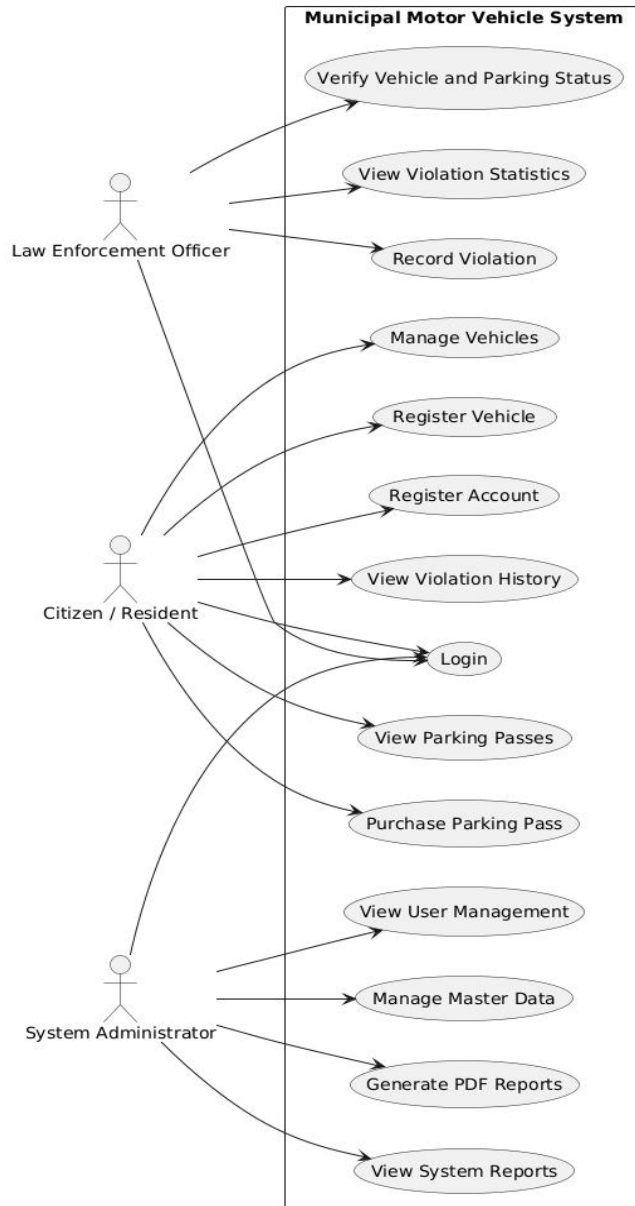
3.2.7 VehicleType & EngineType

- **Purpose:** Enumeration master data for vehicle classification

- **VehicleType Values:** CAR, MOTORBIKE
- **EngineType Values:** ELECTRIC, HYBRID, ICE
- **Usage:** Applied in vehicle registration, violation recording, and reporting for policy differentiation

4. Use Case Analysis

4.1 Use Case Diagram



4.2 Use Case: UC1 - User Registration

Use Case Name: User Registration

Description: A new citizen or law enforcement officer registers an account in the system to gain access to role-specific features.

Authorized Actors:

- Unregistered Citizens
- Unregistered Law Enforcement Personnel
- System (for user validation)

Preconditions:

- User has not previously registered
- Username and email are not already in use

Main Flow:

1. User navigates to registration page
2. User enters: username, password, email, full name, district, and role
3. System validates input (no duplicates, valid email format)
4. System creates User record in database
5. System redirects user to login page
6. User receives confirmation message

Postconditions:

- New User account is created in database
- User can now login with credentials
- Role-based permissions are assigned

Associated REST Endpoints:

- POST /api/auth/register
- Submit registration form
- **Request:** json:
 - {
 "username": "john_citizen",
 "password": "secure_password",
 "email": "john@example.com",
 "fullName": "John Smith",
 "district": "D1",
 "role": "CITIZEN"
 }
- **Response** (201 Created): json
 - {
 "id": 5,
 "username": "john_citizen",
 "password": "secure_password",
 "email": "john@example.com",
 "fullName": "John Smith",
 "district": "D1",

```
"role": "CITIZEN"  
}
```

Alternative Flows:

- **AF1:** Username already exists → Error message displayed
- **AF2:** Email already exists → Error message displayed
- **AF3:** Invalid email format → Validation error displayed

User Story: As a new resident, I want to register an account in the municipality system so that I can manage my vehicles and purchase parking permits without visiting the municipal office.

4.2 Use Case: UC2 - Register Vehicle & Purchase Parking Pass

Use Case Name: Register Vehicle and Purchase Parking Pass

Description: A citizen registers a new personal vehicle and immediately purchases a parking pass to enable immediate use.

Authorized Actors:

- Authenticated Citizens
- System (for validation and persistence)

Preconditions:

- User is authenticated and logged in
- User has not registered this license plate previously
- Selected parking zone exists in system

Main Flow:

1. Citizen navigates to Vehicles section
2. Citizen clicks "Register New Vehicle"
3. Citizen enters: license plate, model, color, vehicle type, engine type
4. System validates license plate is unique
5. System persists vehicle record associated with citizen
6. System displays success message and new vehicle appears in list
7. Citizen optionally navigates to Passes section
8. Citizen clicks "Purchase New Pass"
9. Citizen selects: zone, license plates to cover (up to 4), pass duration
10. System validates user doesn't already have active pass for this zone
11. System creates ParkingPass record with issue date and calculated expiry date
12. System returns pass details with confirmation

Postconditions:

- Vehicle is registered in citizen's inventory
- ParkingPass is created and active

- License plates on pass are valid for parking in selected zone
- Citizen can now park vehicles in specified zone

Associated REST Endpoints:

- POST /api/vehicles
 - Register new vehicle
 - **Request:** json
 - {


```

          "licensePlate": "AB123CD",
          "model": "Tesla Model 3",
          "color": "White",
          "vehicleType": "CAR",
          "engineType": "ELECTRIC",
          "ownerId": 5
          
```
 - **Response** (201 Created): json
 - {


```

          "id": 12,
          "licensePlate": "AB123CD",
          "model": "Tesla Model 3",
          "color": "White",
          "vehicleType": "CAR",
          "engineType": "ELECTRIC",
          "ownerId": 5
          
```
- GET /api/vehicles?userId=5
 - Retrieve user's vehicles
 - **Response** (200 OK):


```

          [
            {
              "id": 12,
              "licensePlate": "AB123CD",
              "model": "Tesla Model 3",
              "color": "White",
              "vehicleType": "CAR",
              "engineType": "ELECTRIC",
              "ownerId": 5
            }
          ]
          
```
- POST /api/passes
 - Purchase parking pass
 - **Request:**

```

          {
            "zone": "Z1",
            "licensePlates": ["AB123CD", "XY789ZZ"],
          
```

- ```

 "ownerId": 5
 }

```
- **Response** (201 Created):
 

```

{
 "id": 8,
 "issueDate": "2026-02-03T10:30:00",
 "expiryDate": "2026-03-03T10:30:00",
 "zone": "Z1",
 "licensePlates": ["AB123CD", "XY789ZZ"],
 "ownerId": 5
}

```
  - GET /api/passes?userId=5
    - Retrieve user's passes
    - **Response** (200 OK):
 

```

[
 {
 "id": 8,
 "issueDate": "2026-02-03T10:30:00",
 "expiryDate": "2026-03-03T10:30:00",
 "zone": "Z1",
 "licensePlates": ["AB123CD", "XY789ZZ"],
 "ownerId": 5
 }
]

```

#### Alternative Flows:

- **AF1:** License plate already registered → Error displayed, suggestion to update existing vehicle
- **AF2:** Active pass already exists for zone → Error displayed, user prompted to renew or change zone
- **AF3:** Invalid vehicle type/engine type → Validation error displayed with available options

**User Story:** > As a citizen with multiple vehicles, I want to register my new electric car and immediately purchase a parking pass so that I can start using the parking zone without delays.

---

### 4.3 Use Case: UC3 - Record Violation & Generate Report

**Use Case Name:** Record Parking Violation and Generate Enforcement Report

**Description:** Law enforcement officers record parking violations in real-time, and administrators generate comprehensive reports for policy analysis.

**Authorized Actors:**

- Authenticated Law Enforcement Officers (recording violations)
- Authenticated Administrators (generating reports)
- System (for validation and aggregation)

#### **Preconditions:**

- Law enforcement officer is authenticated and logged in as LAW\_ENFORCEMENT
- Administrator is authenticated and logged in as ADMIN
- Violation event has occurred in the field

#### **Main Flow - Recording Violation:**

1. Law enforcement officer opens mobile/desktop application
2. Officer enters violation details: license plate, district, vehicle type, engine type
3. System automatically captures current timestamp
4. System validates vehicle type and engine type values
5. System persists violation record to database
6. System returns confirmation with violation ID
7. Officer can immediately record additional violations

#### **Main Flow - Generating Report:**

1. Administrator navigates to Reports section
2. Administrator selects report type: "Violation Statistics"
3. Administrator selects grouping: By District / By Vehicle Type / By Engine Type
4. Administrator optionally selects time period: All time / Last day / Last week / Last month
5. System queries violations matching criteria
6. System aggregates violations by selected grouping
7. System displays dashboard with charts and statistics
8. Administrator clicks "Export as PDF"
9. System generates professional PDF report with:
  - Report title and generation timestamp
  - Summary statistics (total violations, breakdown by group)
  - Detailed table with violation counts by category
  - Municipality branding
10. System provides PDF for download

#### **Postconditions:**

- Violation record is permanently stored with audit trail
- Report data is immediately available for dashboard display
- PDF report can be shared with municipal leadership

#### **Associated REST Endpoints:**

- POST /api/violations
- Record new violation (simulated in demo)
- **Request:** json



- {
  - "district": "D2",
  - "vehicleType": "CAR",
  - "engineType": "ICE",
  - "licensePlate": "XY789ZZ"
- }
- **Response** (201 Created): json
- {
  - "id": 12,
  - "timestamp": "2026-02-03T14:22:15",
  - "district": "D2",
  - "vehicleType": "CAR",
  - "engineType": "ICE",
  - "licensePlate": "XY789ZZ"
- }
- GET /api/reports/violations?period=month&groupBy=district
  - Get violation report
  - **Response** (200 OK):
    - {
    - "D1": 23,
    - "D2": 18,
    - "D3": 31,
    - "D4": 15,
    - "D5": 27
    - }
- GET /api/reports/violations/pdf
  - Download PDF report
  - **Response** (200 OK with PDF binary data)
    - Content-Type: application/pdf
    - Headers: Content-Disposition: attachment; filename="violation\_report\_2026-02-03.pdf"
- GET /api/passes/count
  - Get active passes counter
  - **Response** (200 OK): 142

### Alternative Flows:

- **AF1:** Invalid license plate format → Validation warning but record still accepted
- **AF2:** No violations match selected criteria → Dashboard shows “No data available”
- **AF3:** Database connection error during report generation → Error message displayed with retry option

### User Story:

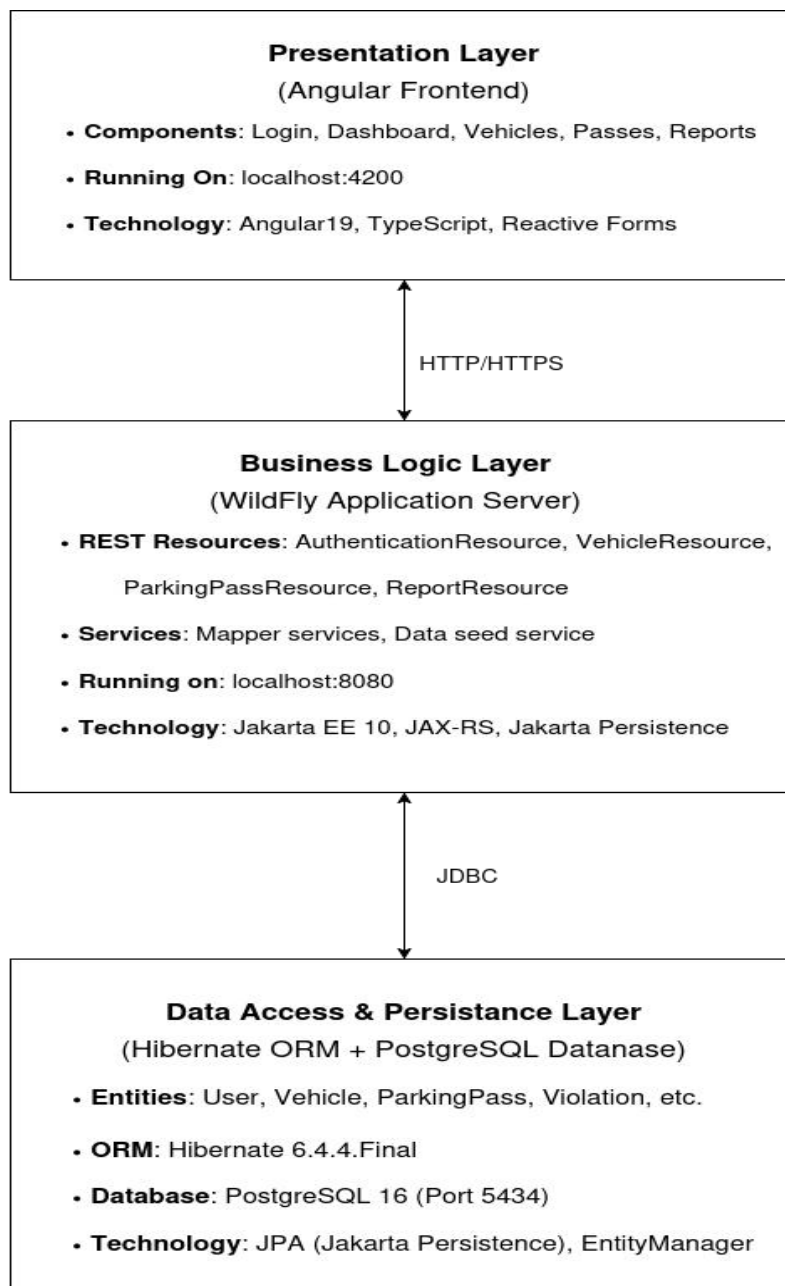
- As a law enforcement officer, I want to quickly record parking violations with accurate timestamp and location so that the enforcement record is complete and enables administrators to analyze violation patterns across the municipality.

- As an administrator, I want to generate detailed violation reports grouped by district and time period so that I can identify problem areas and allocate enforcement resources more effectively.

---

## 5. Architectural Considerations

### 5.1 Layered Architecture Overview



The Municipality Motor Vehicle System implements a **three-tier client-server architecture** with clear separation of concerns:

## 5.2 Layer Responsibilities

### 5.2.1 Presentation Layer (Frontend)

**Technology:** Angular 19 with TypeScript

**Responsibilities:**

- User Interface rendering and interactivity
- Form input validation and user feedback
- HTTP request/response handling via Angular services
- Session state management and authentication tokens
- Real-time dashboard updates via Observable/RxJS

**Key Components:**

- LoginComponent: Authentication UI
- DashboardComponent: Main application shell with child route outlets
- VehiclesComponent: Vehicle CRUD operations
- PassesComponent: Parking pass management
- SummaryComponent: Real-time dashboard with statistics
- BackOfficeComponent: Administrative features (ADMIN role only)

**Cross-Cutting Concerns:**

- CORS handling via CorsFilter in backend
- Error handling and HTTP status code interpretation
- Reactive data binding for real-time updates

### 5.2.2 Business Logic Layer (Backend REST Resources)

**Technology:** Jakarta EE 10 with JAX-RS, WildFly 39.0.0

**Responsibilities:**

- HTTP request processing and routing
- Business rule validation and enforcement
- Transaction management and data consistency
- Error handling and response formatting
- Security and access control

**Key REST Resources (@RequestScoped, @Path):**

- AuthenticationResource (@Path("/auth")): Login and registration endpoints
- VehicleResource (@Path("/vehicles")): Vehicle CRUD operations
- ParkingPassResource (@Path("/passes")): Pass purchase and verification
- ReportResource (@Path("/reports")): Violation reporting and PDF generation
- MasterDataResource (@Path("/masterdata")): District/zone/type data
- DataSeedService: Automatic seeding on startup

**Transactional Boundaries:**

- All write operations (@POST, @PUT, @DELETE) use @Transactional
- Read operations (@GET) don't require transaction but use eager loading

### Error Handling:

```
// Standard response pattern
return Response.status(Response.Status.BAD_REQUEST)
 .entity("{\"error\": \"descriptive message\"}")
 .build();
```

### 5.2.3 Data Persistence Layer (Hibernate + PostgreSQL)

**Technology:** JPA with Hibernate ORM, PostgreSQL 16

### Responsibilities:

- Object-Relational mapping
- Database query execution
- Connection pooling and resource management
- Data integrity via constraints and relationships
- ACID compliance for transactions

### Key Design Patterns:

#### 1. DTO (Data Transfer Object) Pattern:

- Entities: User, Vehicle, ParkingPass, Violation
- DTOs: UserDTO, VehicleDTO, ParkingPassDTO
- Purpose: Decouple API contracts from internal entity structures

#### 2. Mapper Pattern:

- UserMapper.toDTO() / UserMapper.toEntity()
- VehicleMapper.toDTO() / VehicleMapper.toEntity()
- ParkingPassMapper.toDTO() / ParkingPassMapper.toEntity()
- Purpose: Centralized entity-DTO conversions

#### 3. Eager Loading Pattern:

```
// Join Fetch prevents N+1 queries
"SELECT DISTINCT p FROM ParkingPass p LEFT JOIN FETCH p.licensePlates"
```

#### 4. Singleton Pattern (implicit in EntityManager):

- @PersistenceContext(unitName = "parkingPU") injects shared EntityManager

### 5.3 RESTful Architecture Principles

The system strictly adheres to REST (Representational State Transfer) principles:

#### 5.3.1 Resource-Oriented Design

Resources are identified by URIs representing entities:

- /api/users - User collection
- /api/vehicles - Vehicle collection
- /api/passes - Parking pass collection
- /api/violations - Violation collection
- /api/reports - Report collection

### 5.3.2 Standard HTTP Methods

| Method | Purpose            | Example                                  |
|--------|--------------------|------------------------------------------|
| GET    | Retrieve resources | GET /api/vehicles?userId=5               |
| POST   | Create resources   | POST /api/vehicles (with JSON body)      |
| PUT    | Update resources   | PUT /api/vehicles/12 (with updated JSON) |
| DELETE | Remove resources   | DELETE /api/vehicles/12                  |

### 5.3.3 Representation Format

All resources use JSON representation:

```
{
 "id": 5,
 "username": "john_citizen",
 "email": "john@example.com",
 "fullName": "John Smith",
 "district": "D1",
 "role": "CITIZEN"
}
```

### 5.3.4 Statelessness

Each HTTP request is independent and self-contained:

- No server-side session state stored between requests
- Authentication info passed via request headers or body
- Client maintains its own session state (e.g., logged-in user ID)

### 5.3.5 HTTP Status Codes

Standard status codes indicate operation results:

- 200 OK - Successful GET/PUT
- 201 Created - Successful POST (resource created)
- 204 No Content - Successful DELETE
- 400 Bad Request - Invalid input or business rule violation
- 401 Unauthorized - Authentication required
- 404 Not Found - Resource doesn't exist
- 500 Internal Server Error - Server error

### 5.3.6 Content Negotiation

```
@Produces(MediaType.APPLICATION_JSON) // Declares JSON output
@Consumes(MediaType.APPLICATION_JSON) // Expects JSON input
```

## 5.4 UML Deployment Diagram

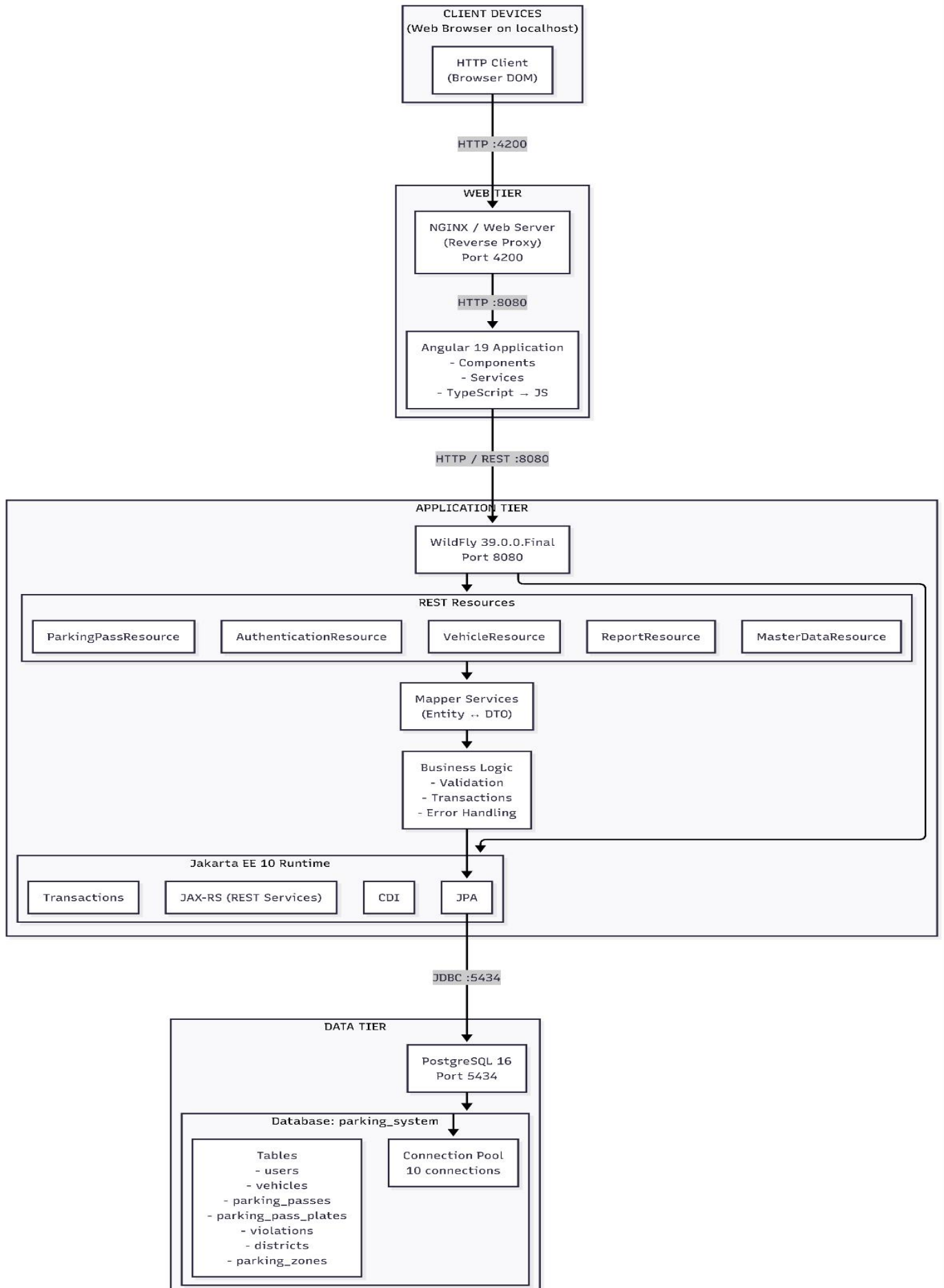
The deployment architecture shows how software components are distributed across hardware nodes.

### Component Details:

| Node       | Technology                 | Port | Purpose                            |
|------------|----------------------------|------|------------------------------------|
| Client     | Browser (Chrome, Firefox)  | 4200 | User interface execution           |
| Web Server | NGINX / Node.js Dev Server | 4200 | Angular app serving                |
| App Server | WildFly 39.0.0.Final       | 8080 | REST API execution, business logic |
| Database   | PostgreSQL 16              | 5434 | Persistent data storage            |

### Communication Protocols:

- **Client ↔ Web Server:** HTTP (port 4200) - Angular app delivery
- **Browser ↔ Backend:** HTTP/REST (port 8080) - JSON data exchange
- **Backend ↔ Database:** JDBC (port 5434) - SQL queries and transactions



---

## 6. Implementation

### 6.1 Backend (Java/Jakarta EE) Architecture

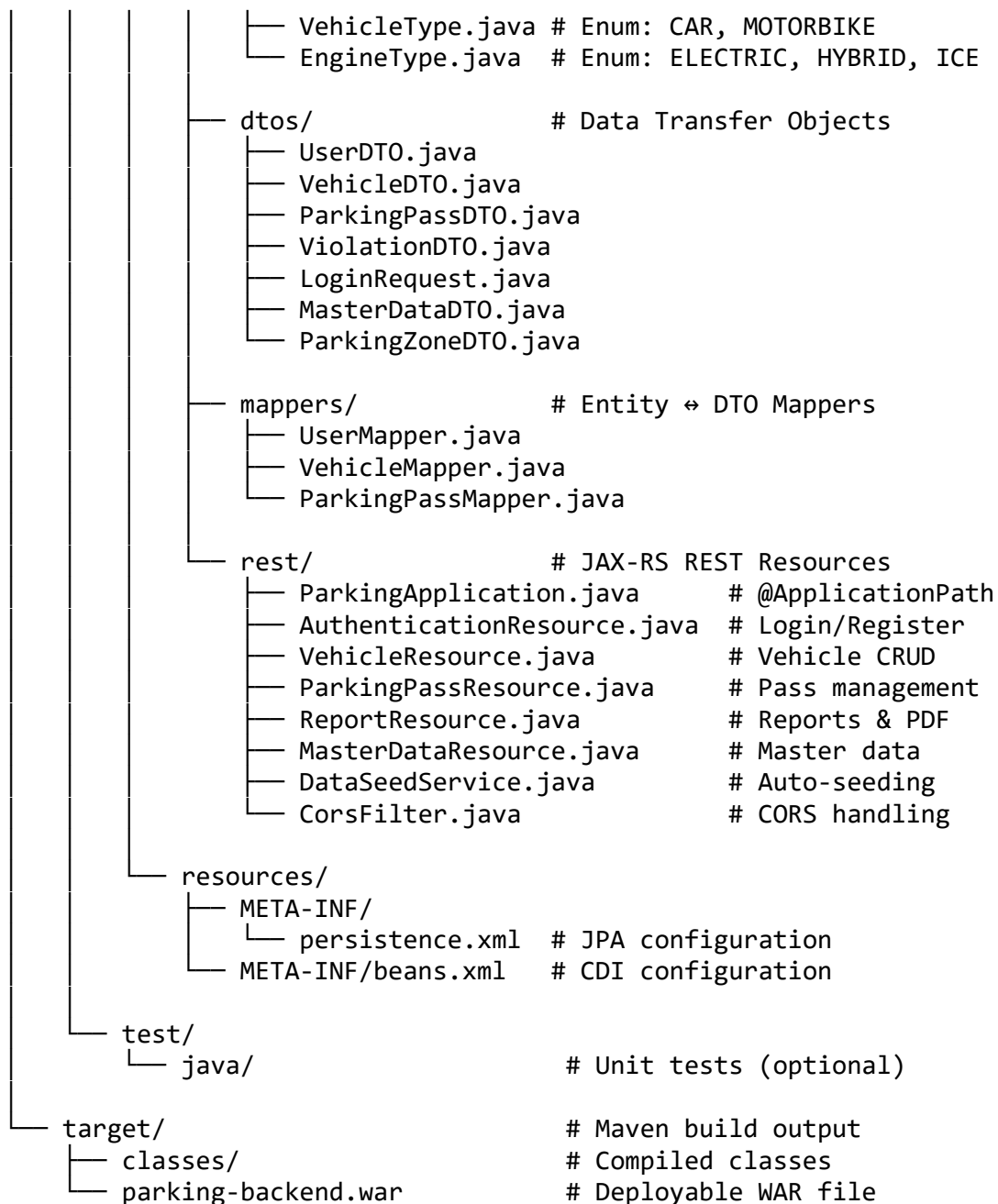
#### 6.1.1 Technology Stack

| Component                 | Technology            | Version      | Purpose                                    |
|---------------------------|-----------------------|--------------|--------------------------------------------|
| <b>Language</b>           | Java                  | 21           | Primary backend programming language       |
| <b>Framework</b>          | Jakarta EE            | 10           | Enterprise application framework           |
| <b>Web Services</b>       | JAX-RS                | 3.1          | REST API implementation                    |
| <b>Persistence</b>        | JPA + Hibernate       | 6.4.4.Final  | Object-Relational Mapping                  |
| <b>Application Server</b> | WildFly               | 39.0.0.Final | Java EE runtime environment                |
| <b>Database Driver</b>    | PostgreSQL JDBC       | 42.7.2       | Database connectivity                      |
| <b>Build Tool</b>         | Apache Maven          | 3.9+         | Build automation and dependency management |
| <b>PDF Generation</b>     | iText                 | OpenSource   | Report generation in PDF format            |
| <b>Logging</b>            | System.out/System.err | -            | Console logging for development            |

#### 6.1.2 Backend Project Structure

```
parking-backend/
├── pom.xml # Maven configuration
├── src/
│ └── main/
│ ├── java/it/unifi/swam/parking/
│ │ └── model/ # JPA Entity Classes
│ │ ├── User.java # User entity with roles
│ │ ├── Vehicle.java # Vehicle entity with types
│ │ ├── ParkingPass.java # Pass entity with zone & plates
│ │ ├── Violation.java # Violation entity
│ │ ├── District.java # District master data
│ │ └── ParkingZone.java # Zone master data
```





### 6.1.3 Key Packages and Components

**Package:** model

- JPA Entity Classes - Each class uses `@Entity` and `@Table` annotations
- Relationships defined with `@ManyToOne`, `@OneToMany`, `@ElementCollection`
- Implements `Serializable` for potential clustering/caching
- Provides constructors and JavaBean-style getters/setters

**Package:** dtos

- Data Transfer Objects
- Lightweight classes for API serialization
- Decouple API contracts from internal entity changes
- Used for both request payloads and response bodies
- Example: `VehicleDTO` contains same fields as `Vehicle` but excludes internal references

### Package: mappers

- Entity/DTO Conversion Services
- Static utility methods for entity-to-DTO conversion
- Example: `VehicleMapper.toDTO(vehicle)` returns a DTO
- Example: `VehicleMapper.toEntity(vehicleDTO, owner)` creates entity with FK
- Centralizes transformation logic for maintainability

### Package: rest

- JAX-RS REST Resources
1. **AuthenticationResource** - User Authentication/Registration
    - `@POST /auth/login` - User login with credentials
    - `@POST /auth/register` - New user registration
    - Uses JPQL queries for user lookup
    - Returns `UserDTO` with role for frontend authorization
  2. **VehicleResource** - Vehicle Lifecycle Management
    - `@GET /vehicles` - List vehicles (filtered by `userId`)
    - `@POST /vehicles` - Register new vehicle
    - `@PUT /vehicles/{id}` - Update vehicle details
    - `@DELETE /vehicles/{id}` - Remove vehicle
    - Enforces ownership validation
  3. **ParkingPassResource** - Parking Permit Management
    - `@GET /passes` - List passes for user with eager loading
    - `@POST /passes` - Purchase new pass with validation
    - `@GET /passes/check/{licensePlate}` - Verify pass validity
    - `@GET /passes/count` - Active pass counter for dashboard
    - Implements business rule: single active pass per zone/user
  4. **ReportResource** - Reporting & Analytics
    - `@GET /reports/violations` - Aggregate violation statistics
    - `@GET /reports/violations/pdf` - Generate PDF report
    - Supports grouping by district, `vehicleType`, `engineType`
    - Supports time-period filtering
    - Uses iText library for professional PDF generation
  5. **MasterDataResource** - System Configuration Data
    - `@GET /masterdata/districts` - List all districts
    - `@GET /masterdata/zones` - List all parking zones
    - `@GET /masterdata/vehicletypes` - Vehicle type options

- @GET /masterdata/engineypes - Engine type options
- Read-only endpoints for frontend dropdowns
- 6. **DataSeedService** - Automatic Database Initialization
  - Fires on application startup
  - Detects empty database
  - Populates districts (D1-D5), zones (Z1-Z4), and master data
  - Uses @javax.ejb.Startup lifecycle hook
- 7. **CorsFilter** - Cross-Origin Resource Sharing
  - Allows frontend (localhost:4200) to call backend (localhost:8080)
  - Adds CORS headers to all responses
  - Essential for local development and cross-domain deployments

---

## 6.2 Frontend (Angular) Architecture

### 6.2.1 Technology Stack

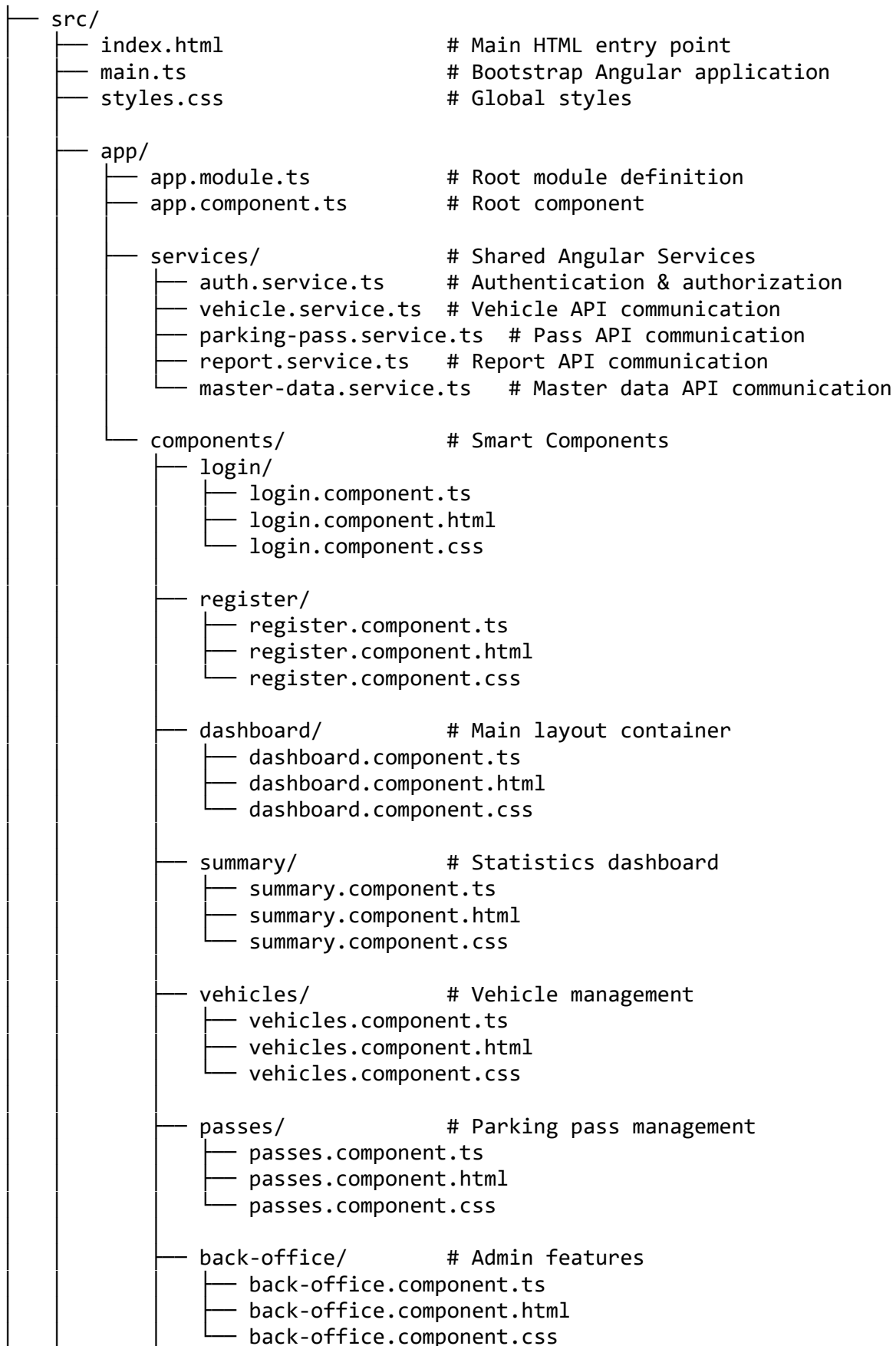
| Component               | Technology                          | Version  | Purpose                             |
|-------------------------|-------------------------------------|----------|-------------------------------------|
| <b>Framework</b>        | Angular                             | 19       | Frontend SPA framework              |
| <b>Language</b>         | TypeScript                          | 5.x      | Type-safe JavaScript superset       |
| <b>Styling</b>          | CSS                                 | 3        | Component styling                   |
| <b>HTTP Client</b>      | HttpClientModule                    | Built-in | REST API communication              |
| <b>Forms</b>            | ReactiveFormsModule,<br>FormsModule | Built-in | Form handling and validation        |
| <b>Routing</b>          | RouterModule                        | Built-in | Client-side navigation              |
| <b>State Management</b> | RxJS Services                       | 7.x      | Reactive data flow with Observables |
| <b>Build Tool</b>       | Angular CLI                         | 19       | Development and build automation    |
| <b>Package Manager</b>  | npm                                 | 10.x     | JavaScript dependency management    |
| <b>Web Server</b>       | Node.js Dev Server                  | 20.x     | Local development server            |

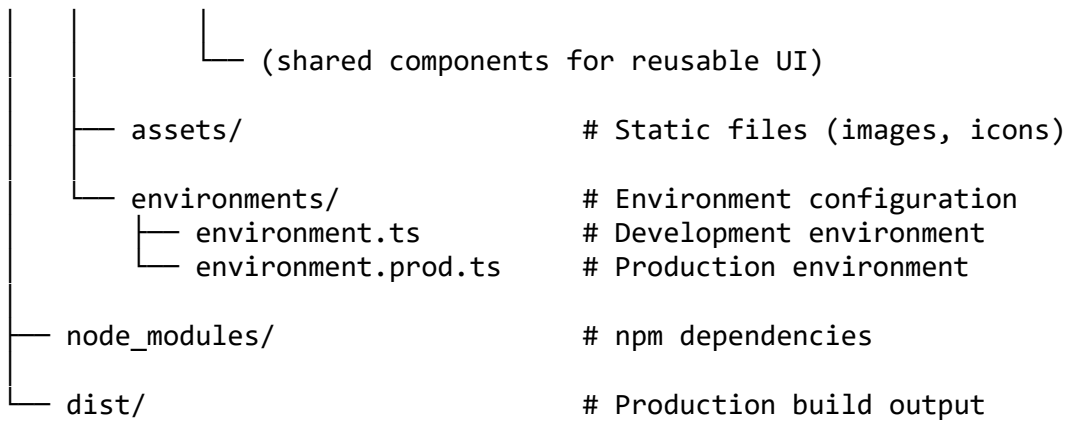
### 6.2.2 Frontend Project Structure

```

parking-frontend/
├── package.json # npm configuration
├── angular.json # Angular CLI configuration
└── tsconfig.json # TypeScript configuration

```





### 6.2.3 Key Packages and Components

#### Services Layer - Encapsulate Backend Communication

##### 1. AuthService - User Authentication

```

login(username: string, password: string): Observable<UserDTO>
register(userDTO: UserDTO): Observable<UserDTO>
getCurrentUser(): UserDTO | null
logout(): void
isLoggedIn(): boolean
getUserRole(): string

```

##### 2. VehicleService - Vehicle Operations

```

getVehicles(userId: number): Observable<VehicleDTO[]>
registerVehicle(vehicleDTO: VehicleDTO): Observable<VehicleDTO>
updateVehicle(id: number, vehicleDTO: VehicleDTO):
Observable<VehicleDTO>
deleteVehicle(id: number): Observable<void>

```

##### 3. ParkingPassService - Pass Management

```

getPasses(userId: number): Observable<ParkingPassDTO[]>
purchasePass(passDTO: ParkingPassDTO): Observable<ParkingPassDTO>
checkPass(licensePlate: string): Observable<PassStatus>
getActivePassCount(): Observable<number>

```

##### 4. ReportService - Reporting & Export

```

getViolationReport(period: string, groupBy: string):
Observable<ViolationReport>
downloadPdfReport(): Observable<Blob>

```

##### 5. MasterDataService - Reference Data

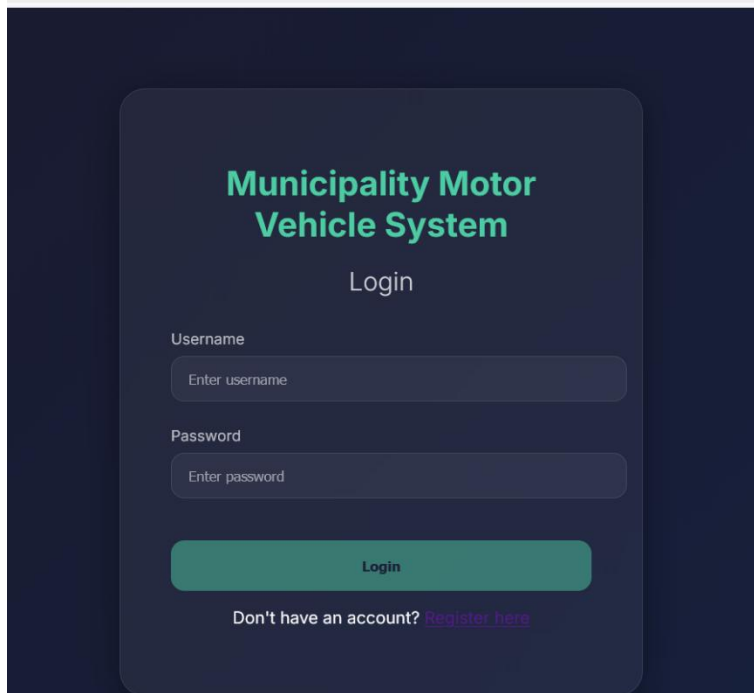
```

getDistricts(): Observable<District[]>
getZones(): Observable<ParkingZone[]>
getVehicleTypes(): Observable<string[]>
getEngineTypes(): Observable<string[]>

```

## Components Layer - UI Presentation & User Interaction

1. **LoginComponent** - Authentication UI
  - Form with username/password fields
  - Calls AuthService.login()
  - Stores user in local storage
  - Navigates to dashboard on success

The image shows a login interface for the 'Municipality Motor Vehicle System'. The title is in green text. Below it is the word 'Login'. There are two input fields: 'Username' with a placeholder 'Enter username' and 'Password' with a placeholder 'Enter password'. Below these is a green 'Login' button. At the bottom, there is a link that says 'Don't have an account? Register here'.

2. **RegisterComponent** - New User Registration
  - Form with fields: username, password, email, fullName, district, role
  - Dropdown selections for district and role
  - Calls AuthService.register()
  - Redirects to login after successful registration

# Municipality Motor Vehicle System

## Register

**Full Name**

**Email**

**Username**

**Password**

**Florence District**

**Role**

**Register**

Already have an account? [Login here](#)

### 3. DashboardComponent - Main Layout

- Navigation sidebar with role-based menu items
- Router outlet for child component display
- User profile display and logout button
- Responsive layout: sidebar + main content area

Municipality Motor Vehicle System

Summary
Vehicles
Parking Passes
Back Office

System Admin  
ADMIN

Logout

Welcome back, System Admin!

Summary

Export PDF Report

Total System Vehicles

6

Total Active Passes

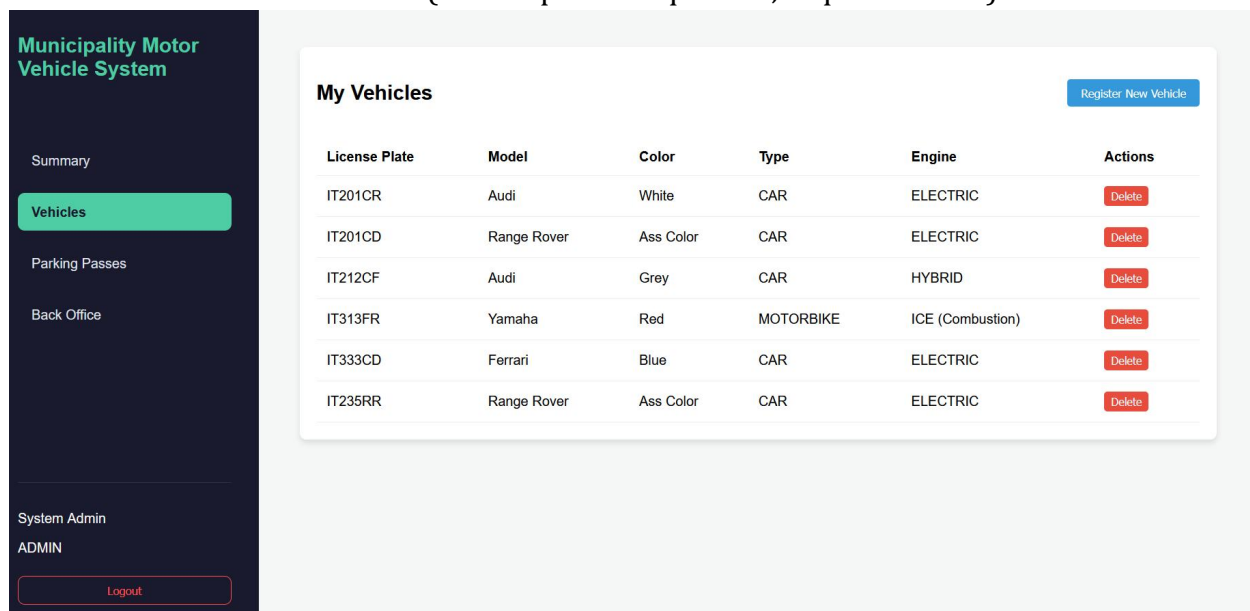
7

System Statistics: Parking Violations

Period: All Time Group By: District

| DISTRICT               | COUNT |
|------------------------|-------|
| D2                     | 1     |
| D3 - Gavinana-Galluzzo | 2     |
| D5                     | 1     |

4. **SummaryComponent** - Real-time Dashboard(The above screenshot also consists of the Summary Component on the right in the main content area).
  - Displays active parking pass count (auto-refreshing)
  - Shows violation statistics by district
  - Charts for visual representation
  - Time period filters (day, week, month, all)
5. **VehiclesComponent** - Vehicle Management
  - Lists user's vehicles in table format
  - "Add Vehicle" button with form modal
  - Edit/Delete buttons for each vehicle
  - Form validation (license plate uniqueness, required fields)



6. **PassesComponent** - Parking Pass Management
  - Lists user's passes with expiry status
  - "Purchase New Pass" button with form
  - Zone selection dropdown
  - License plate multi-select (up to 4 plates)
  - Pass validity indicator (Active/Expired)



Municipality Motor Vehicle System

Summary
Vehicles
Parking Passes
Back Office
System Admin
ADMIN
Logout

Parking Passes

Purchase New Pass

| License Plate                            | Zone        | Issue Date       | Expiry Date      | Status |
|------------------------------------------|-------------|------------------|------------------|--------|
| IT201CD<br>IT212CF<br>IT333CD<br>IT313FR | D4 - Center | 1/27/26, 1:12 AM | 2/27/26, 1:12 AM | ACTIVE |
| IT333CD<br>IT201CD<br>IT212CF<br>IT313FR | D2 - Center | 1/27/26, 1:01 AM | 2/27/26, 1:01 AM | ACTIVE |
| IT212CF<br>IT313FR<br>IT201CD<br>IT333CD | D5 - Center | 1/27/26, 1:11 AM | 2/27/26, 1:11 AM | ACTIVE |
| IT333CD<br>IT201CD<br>IT212CF<br>IT313FR | D1 - Center | 1/27/26, 1:00 AM | 2/27/26, 1:00 AM | ACTIVE |
| IT313FR<br>IT201CD<br>IT212CF<br>IT333CD | D4 - Center | 1/27/26, 1:12 AM | 2/27/26, 1:12 AM | ACTIVE |
| IT201CR                                  | ZONE_C      | 1/26/26, 1:21 AM | 2/26/26, 1:21 AM | ACTIVE |

## 7. BackOfficeComponent - Administrative Features

- Visible only to users with ADMIN role
- Master data management (districts, zones)
- Violation report generation
- PDF report download button
- Report grouping and filtering options

Municipality Motor Vehicle System

Summary
Vehicles
Parking Passes
Back Office
System Admin
ADMIN
Logout

Back-Office Data Management

Vehicle Types

New Vehicle Type (e.g. CAR)
Add

CAR
MOTORBIKE

Engine Types

New Engine Type (e.g. ELECTRIC)
Add

ICE (Combustion)
ELECTRIC
HYBRID

Districts (Florence)

District Name (e.g. D1)
Description (Characteristics/Neighborhoods)
Add District

D1 - Historic Center
D2 - Campo di Marte
D3 - Gavinana-Galluzzo
D4 - Isolotto-Legnaia
D5 - Rifredi

Parking Zones

Zone Code (e.g. Z1)
Select District
Add

## 7. Summary

The **Municipality Motor Vehicle System** represents a modern, production-ready implementation of a client-server architecture using industry-standard technologies:

### 1.7.1 Key Architectural Achievements:

1. **Clear Separation of Concerns:** Presentation, Business Logic, and Persistence layers independently , deployable and testable
2. **RESTful Design:** Stateless HTTP-based communication with standard methods and status codes
3. **Scalability:** Support for horizontal scaling through stateless backend design
4. **Maintainability:** Centralized configuration, comprehensive logging, and error handling
5. **User Experience:** Real-time dashboards, responsive UI, and role-based access control

### 1.7.2 Technical Excellence:

- Eager loading queries prevent N+1 performance issues
- Transactional boundaries ensure data consistency
- CORS configuration enables secure cross-origin communication
- Automatic database seeding reduces deployment friction
- Professional PDF reporting demonstrates advanced backend capabilities

### 1.7.3 Challenge

Bridging the Cyber-Physical Gap for System Enforcement: Designing the system required accounting for real-world limitations where software alone cannot enforce physical rules . We had to define clear administrative and police roles within the application to handle the manual verification of citizen residency documentation and the physical enforcement of digital parking passes on the streets.

This technical report demonstrates the system's readiness for production deployment in a modern municipal environment.

---